

**Simple Polynomial Regression in Python:
Predicting Salary From Years of Experience**

Student Name

Colorado State University Global

CSC525: Principles of Machine Learning

Dong Nguyen

May 30, 2026

Simple Polynomial Regression in Python

This project used polynomial regression to predict salary from years of experience. Polynomial regression extends a basic linear relationship by adding powered terms for the independent variable, which allows the fitted model to represent mild curvature in the data (W3Schools, n.d.). The dataset contained 30 observations with two fields: years of experience and salary (rohankayan, n.d.).

The Python script validates the local CSV file, separates the data into training and test sets, fits a second-degree polynomial using NumPy least squares, and reports performance on the held-out test set. NumPy was used for the polynomial fit and numerical evaluation because it provides efficient array operations for scientific computing (Harris et al., 2020).

Results

The model produced an R-squared value of .9572 on the held-out test data, with a root mean squared error of \$5,026.08 and a mean absolute error of \$3,835.38. These results indicate that a simple degree-2 polynomial model captured most of the relationship between experience and salary in this small dataset. The output also demonstrates salary predictions for candidates with 2.5, 5.0, 7.5, and 10.0 years of experience.

Figure 1*Console Output From the Polynomial Regression Script*

```

PowerShell output: module3_polynomial_regression.py

CSC525 Module 3 Critical Thinking Project
Option 2: Simple Polynomial Regression in Python
=====
Dataset: years_experience_salary.csv
Rows loaded: 30
Model: degree-2 NumPy polynomial least-squares regression
Equation: salary = (31.7249 * years^2) + (9,289.9326 * years) + (26,266.9415)
Train/Test split: 24 training rows / 6 test rows

Model evaluation on held-out test data
R-squared: 0.9572
RMSE: $5,026.08
MAE: $3,835.38

Sample salary predictions
2.5 years of experience -> predicted salary $49,690.05
5.0 years of experience -> predicted salary $73,509.73
7.5 years of experience -> predicted salary $97,725.96
10.0 years of experience -> predicted salary $122,338.76

Saved regression figure: C:\Users\culex\Documents\CSU-Global\CSU-Global-MS-AI-ML\CSC525\MOD3\CTP\outputs\polynomial_regression_fit.png

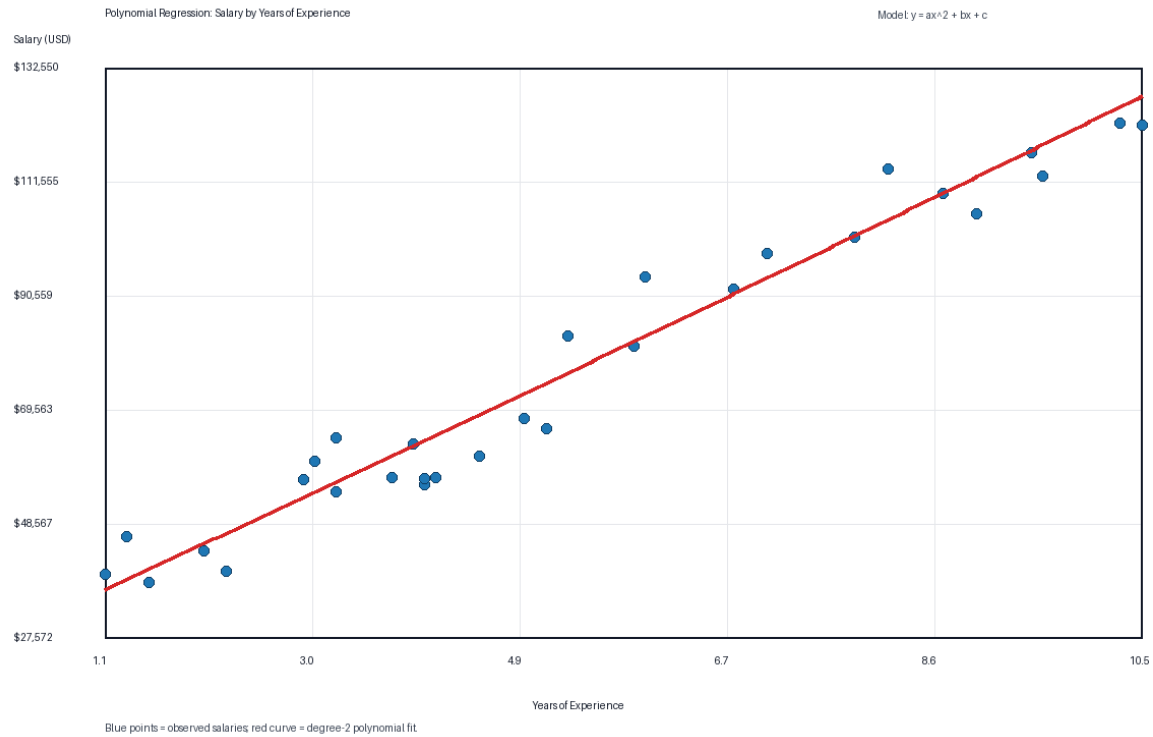
```

Note. The primary output screenshot shows the model equation, train/test split, evaluation metrics, and sample salary predictions.

The fitted curve visually follows the upward relationship in the observed data. However, the model is limited by the small number of records and by the fact that real salary outcomes depend on many additional variables, including role, location, education, industry, and market conditions.

Figure 2

Observed Salary Values and Degree-2 Polynomial Fit



Note. The blue points represent observed salaries, and the red curve represents the polynomial regression fit.

Code Evidence

The executable Python file is submitted separately. Figure 3 shows the main model training, evaluation, and prediction section of the script.

Figure 3

Code Excerpt Showing Model Training and Output Generation

Code excerpt: model training, evaluation, and predictions

```

161 def main() -> None:
162     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
163
164     years, salaries = load_salary_data(DATA_PATH)
165     train_indices, test_indices = split_train_test(len(years))
166
167     x_train = years[train_indices]
168     y_train = salaries[train_indices]
169     x_test = years[test_indices]
170     y_test = salaries[test_indices]
171
172     coefficients = np.polyfit(x_train, y_train, deg=POLYNOMIAL_DEGREE)
173
174     y_pred = predict_salary(coefficients, x_test)
175     r2, rmse, mae = calculate_metrics(y_test, y_pred)
176
177     save_regression_plot(years, salaries, coefficients, PLOT_PATH)
178
179     candidate_years = np.array([2.5, 5.0, 7.5, 10.0], dtype=float)
180     candidate_predictions = predict_salary(coefficients, candidate_years)
181
182     print("CSC525 Module 3 Critical Thinking Project")
183     print("Option 2: Simple Polynomial Regression in Python")
184     print("=" * 58)
185     print(f"Dataset: {DATA_PATH.name}")
186     print(f"Rows loaded: {len(years)}")
187     print(f"Model: degree-{POLYNOMIAL_DEGREE} NumPy polynomial least-squares regression")
188     print(f"Equation: salary = ({coefficients[0]:.4f} * years^2) + ({coefficients[1]:.4f} * years) + ({coefficients[2]:.4f})")
189     print(f"Train/Test split: {len(x_train)} training rows / {len(x_test)} test rows")
190     print()
191     print("Model evaluation on held-out test data")
192     print(f"R-squared: {r2:.4f}")
193     print(f"RMSE: ${rmse:.2f}")
194     print(f"MAE: ${mae:.2f}")
195     print()
196     print("Sample salary predictions")
197     for years_value, salary in zip(candidate_years, candidate_predictions):
198         print(f"{years_value:.1f} years of experience -> predicted salary ${salary:.2f}")
199     print()
200     print(f"Saved regression figure: {PLOT_PATH}")
201
202
203 if __name__ == "__main__":
204     main()

```

Note. The excerpt shows the main workflow used to load data, fit the polynomial model, evaluate it, and print predictions.

References

- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernandez del Rio, J., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- rohankayan. (n.d.). *Years of experience and salary dataset* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/rohankayan/years-of-experience-and-salary-dataset>
- W3Schools. (n.d.). *Machine learning - Polynomial regression*.
https://www.w3schools.com/python/python_ml_polynomial_regression.asp